TinkerWithMe

tinkerwithanne@gmail.com
Nairobi, Kenya

Smart City · Full Curriculum

Full-day camp (5 hrs) · 10–14 yrs · Intermediate

Full facilitator curriculum **PREMIUM**

Overview

Teams of 3–4 each build one city subsystem, then connect them on a shared city board.

DETAIL	
Track	Arduino Robotics & Electronics
Format	Full-day camp (5 hrs)
Ages	10–14 yrs
Level	Intermediate
Price	from KES 7,000
Standalone	Yes
Prerequisites	None — standalone

Course Overview

Smart City is a one-day, hands-on Arduino camp where four teams of 3–4 students each engineer a different subsystem of a working miniature city — street lights, a traffic intersection, a parking garage, and a weather station. Every team builds, wires and codes their own Arduino Uno module in parallel, then brings it to a shared city board where all four subsystems sit side by side and "come to life" together for a final showcase.

The day is designed as an arc: it opens with a shared briefing on how sensors and actuators let a city "sense and respond" to its environment, moves into focused, guided build time for each subsystem (photoresistors and LEDs, multi-light state machines, ultrasonic counting, and environmental sensing with an LCD), and closes with an integration and demo phase where teams physically place their modules

on the board, narrate what their subsystem does, and run the "city" together in front of the group.

By the end of the day, each student takes home a working, battery/USB-powered subsystem module they helped build and code, plus a group photo of the entire assembled city board in action. The take-home module is intentionally left fully wired on its breadboard so students can keep experimenting, demo it to family, or bring it back for future camps.

Learning Outcomes

- Read analog sensors (photoresistor/LDR, DHT11) and convert raw sensor values into meaningful decisions in code.
- Design and wire multi-component circuits on a breadboard, including correct use of current-limiting resistors and pull-up/pull-down resistors.
- Build a non-blocking state machine using `millis()` instead of `delay()`, and understand why blocking code breaks multi-part systems.
- Use an external interrupt to respond instantly to a button press (pedestrian crossing) without polling.
- Measure distance with an ultrasonic sensor and use sensor transitions (not raw readings) to count events reliably.
- Drive common output devices — LEDs, a servo motor, a 4-digit numeric display, and an I2C LCD — from Arduino code.
- Collaborate in a small engineering team to plan, divide, build, debug and present a working physical system on a deadline.

Materials Master List

ITEM	SPEC	QTY PER GROUP/ STUDENT	NOTES
Arduino Uno R3 (or compatible)	ATmega328P board	1 per team (4 total)	Nerokas/Pixel Electronics stock these; label each board with team name using tape
USB cable	USB-A to USB-B, 1m	1 per team	For programming and power
Breadboard	830 tie-point, full size	1 per team	

			Street Lights team may need 2 if using potentiometer + many LEDs
Jumper wire pack	Male-male, male-female, assorted lengths	1 pack (40+) per team	Buy in bulk, distribute from a shared bin
Resistors 220Ω	1/4W	10 per team	For all LEDs
Resistors 10kΩ	1/4W	4 per team	Voltage dividers, pull-downs
LEDs 5mm	Assorted: red, yellow/amber, green	Street Lights: 4 white/yellow; Traffic: 8 (red/yellow/green x2 sets + ped red/green); Parking: 2 (red, green)	Diffused lenses look best from a distance
Photoresistor (LDR)	5mm GL5528 or similar	1 (Street Lights), 1 (Weather Station)	Pair each with a 10kΩ resistor
Potentiometer	10kΩ linear, breadboard-mount	1 (Street Lights)	Used as adjustable darkness threshold
Push button	12mm tactile momentary switch	1 (Traffic System)	Pedestrian crossing request
Passive buzzer	5V piezo buzzer	1 (Traffic System)	Driven with `tone()` / `noTone()`
Ultrasonic sensor	HC-SR04	2 (Parking Garage)	One at entry, one at exit
Micro servo motor	SG90 9g	1 (Parking Garage)	Acts as the barrier arm; needs a popsicle-stick arm glued on
4-digit 7-segment display module	TM1637-driven, common breakout	1 (Parking Garage)	Requires TM1637Display library
DHT11 sensor module	3-pin breakout with onboard pull-up	1 (Weather Station)	Requires Adafruit DHT + Unified Sensor libraries
16x2 LCD with I2C backpack	I2C address 0x27 or 0x3F	1 (Weather Station)	Run an I2C scanner sketch first if the address is unknown
Power source	USB power bank (10,000mAh) OR 4-port USB charging hub + power strip	1 per team, or 1 shared hub	Keeps boards running once disconnected from laptops during integration
Laptop with Arduino IDE installed	Arduino IDE 2.x, all libraries pre-installed	1 per team	Pre-install libraries before camp day: Servo (built-in), TM1637Display, DHT sensor library, Adafruit Unified Sensor, LiquidCrystal_I2C

Large city board	Foam board or plywood, approx. 90cm x 120cm	1 shared	Pre-drawn or painted with roads/ intersections/parking bay outlines
Craft materials	Small cardboard boxes, coloured paper, popsicle sticks, toy cars (4–6), green/grey paint or paper for grass/roads	1 shared kit	For buildings, lamp posts, and gate arms
Hot glue gun + sticks	Low-temp, with adult/ facilitator supervision	2 shared	For attaching craft elements to modules and board
Wire strippers/ cutters	Small craft/electronics pair	2 shared	For trimming jumper wires if needed
Masking tape, marker pens, sticky labels	—	1 shared kit	For labelling pins, teams, and boards
Multi-socket power strip / extension cable	4–6 outlets	1–2 shared	For charging laptops and powering USB hub

Session Plans

All four sessions run concurrently as build stations — each team is assigned one subsystem and works through its own lesson below, with one facilitator circulating between stations (or one facilitator per station if staffing allows). Each session plan is written for a build block of approximately 140 minutes, matching the main construction phase of the full-day schedule (see Assessment & Showcase for how the day fits together).

1. Street Lights — Photoresistor-Activated LEDs

Objectives

- Explain how an LDR changes resistance with light and how a voltage divider turns that into a readable analog value.
- Wire and code a system where LEDs turn on automatically when ambient light drops below an adjustable threshold.
- Use `analogRead()`, `map()` and a potentiometer to build a calibratable sensor system.

Timing

- 0–10 min: Hook — facilitator dims classroom lights/covers an LDR with a hand while displaying live Serial Monitor values on a projector, asking "what do you notice?"
- 10–25 min: Explain voltage dividers with a whiteboard diagram; explain the role of the 10kΩ resistor and why analog pins read 0–1023.
- 25–70 min: Team builds the breadboard circuit (LDR divider, potentiometer, 4 LEDs with resistors) following the wiring list below.
- 70–100 min: Facilitator live-codes the first draft with the team typing along; upload and test with Serial Monitor open.
- 100–120 min: Teams calibrate the potentiometer threshold to match the room, and test with a torch/phone flashlight to simulate "night".
- 120–140 min: Debug, tidy wiring, mount LEDs onto popsicle-stick "lamp posts" with hot glue ready for the city board.

Step-by-step

1. Ask: "How does a street light know when to turn on?" Draw out that it's light level, not a clock.
2. Show the LDR component; pass it around, have students test its resistance conceptually by covering it with a hand.
3. Draw the voltage divider on the whiteboard: 5V → LDR → junction (to A0) → 10kΩ resistor → GND. Explain that when it's dark, LDR resistance is high, so more voltage appears at the junction — the reading goes up (clarify this against their intuition, since often "darker = lower reading" is the common misconception).
4. Distribute breadboards and components. Supervise wiring of the LDR divider first; verify with a multimeter or by uploading a bare analogRead+Serial.print sketch before adding LEDs.
5. Add the potentiometer as a second voltage divider into A1 — explain it lets the team "tune" the threshold live without reflashing code.
6. Wire the four LEDs with 220Ω resistors into digital pins 8–11.
7. Type in the full sketch together, explaining each block, then upload.
8. Test with room lights on/off and with a torch. Have the team write down their chosen threshold range for their showcase notes.

9. In the last 20 minutes, help the team glue LEDs onto craft "lamp post" sticks so they're ready to stand on the city board.

Build detail

Materials: Arduino Uno, breadboard, LDR, 10kΩ resistor, 10kΩ potentiometer, 4x LED (5mm), 4x 220Ω resistor, jumper wires, 4x popsicle sticks, hot glue.

Wiring (component → pin):

- LDR leg 1 → 5V; LDR leg 2 → A0 and → one leg of 10kΩ resistor → GND (voltage divider)
- Potentiometer outer legs → 5V and GND; wiper (middle pin) → A1
- LED 1 anode → 220Ω resistor → Pin 8; cathode → GND
- LED 2 anode → 220Ω resistor → Pin 9; cathode → GND
- LED 3 anode → 220Ω resistor → Pin 10; cathode → GND
- LED 4 anode → 220Ω resistor → Pin 11; cathode → GND

```
// Smart City - Street Lights
// Reads an LDR (light sensor) and turns on street lamp LEDs when it gets dark.
// A potentiometer lets the team tune exactly how "dark" is dark enough.

const int ldrPin = A0;                  // LDR voltage divider output
const int potPin = A1;                  // Potentiometer wiper - sets darkness threshold
const int lampPins[] = {8, 9, 10, 11};  // 4 street lamp LEDs
const int numLamps = 4;

void setup() {
  Serial.begin(9600);
  for (int i = 0; i < numLamps; i++) {
    pinMode(lampPins[i], OUTPUT);
  }
}

void loop() {
  int lightLevel = analogRead(ldrPin);   // 0 (dark) - 1023 (bright), depends on wiring
  int rawPot = analogRead(potPin);
  int threshold = map(rawPot, 0, 1023, 100, 700); // adjustable threshold range

  bool isNight = lightLevel < threshold;

  for (int i = 0; i < numLamps; i++) {
    digitalWrite(lampPins[i], isNight ? HIGH : LOW);
  }

  // Debug info so the team can calibrate the potentiometer live
  Serial.print("Light level: ");
  Serial.print(lightLevel);
  Serial.print("  Threshold: ");
  Serial.print(threshold);
  Serial.print("  Lamps: ");
```

```
  Serial.println(isNight ? "ON (night)" : "OFF (day)");

  delay(200); // small delay keeps the Serial Monitor readable
}
```

Checks for understanding

- Why do we need the 10kΩ resistor in the LDR circuit — what would happen without it?
- If the Serial Monitor shows a HIGHER number when you cover the LDR with your hand, what does that tell you about how it's wired?
- What does the potentiometer actually change in this program — the sensor, or the decision the code makes?

MISTAKE	FIX
LDR reading is always 0 or always 1023, never changes	Check the voltage divider — LDR and 10kΩ resistor must be in series between 5V and GND, with the middle junction going to A0
LEDs stay on all the time regardless of light	Threshold from the potentiometer may be set too high; turn the potentiometer or check the `map()` range in code
LED very dim or not lighting	Check LED polarity (longer leg = anode, toward the resistor) and confirm the resistor is actually in that leg's path

2. Traffic System — Multi-Intersection Lights with Pedestrian Crossing

Objectives

- Design a non-blocking state machine using `millis()` to run two sets of traffic lights safely without ever showing green on both roads at once.
- Use an external interrupt to capture a pedestrian button press instantly, even while lights are mid-sequence.
- Practise breaking a real-world system (a road intersection) into clearly named states and transitions.

Timing

- 0–10 min: Hook — ask the team to act out being cars and a pedestrian at a real intersection; identify all the "states" a light can be in.
- 10–25 min: Whiteboard the state diagram together (NS Green → NS Yellow → All Red → EW Green → EW Yellow → All Red → repeat, with a pedestrian branch).
- 25–30 min: Explain why `delay()` would break this (can't check the button while "frozen" in a delay) and introduce `millis()` timing.

- 30–75 min: Wire the 8 LEDs, button, and buzzer per the wiring list.
- 75–115 min: Type and upload the full sketch together, explaining the `enum` states, the `switch` statement, and the interrupt function.
- 115–135 min: Test full cycle; press the button mid-cycle and confirm the pedestrian phase is honoured at the next safe point.
- 135–140 min: Tidy wiring and mount LED sets onto small cardboard traffic-light housings for the city board.

Step-by-step

1. Role-play the intersection with students as "cars" — establish that both roads can never be green together.
2. Draw the full state diagram on the whiteboard with arrows and timings before touching code.
3. Introduce interrupts conceptually: "the button press doesn't wait its turn — it interrupts whatever the Arduino is doing to flag itself immediately."
4. Guide the wiring: two 3-LED sets (red/yellow/green), two pedestrian LEDs, one button on pin 2 (the only interrupt-capable pin easily available on Uno alongside pin 3), one buzzer.
5. Build the sketch in layers: first get the NS/EW cycle running with `millis()`, test it, then add the button interrupt and pedestrian phase.
6. Test edge cases together: press the button repeatedly fast (debounce should prevent double-triggering), press it during a green phase (request should queue until the next all-red).
7. Ask the team to explain the state diagram back to you before moving on to enclosure building.

Build detail

Materials: Arduino Uno, breadboard, 8x LED (2 sets of red/yellow/green + 1 pedestrian red + 1 pedestrian green), 8x 220Ω resistor, 1x push button, 1x passive buzzer, jumper wires, small cardboard housings.

Wiring (component → pin):

- Pedestrian button → Pin 2 (other leg to GND; use `INPUT_PULLUP` in code, no external resistor needed)

- NS Red LED → 220Ω → Pin 4; NS Yellow LED → 220Ω → Pin 5; NS Green LED → 220Ω → Pin 6
- EW Red LED → 220Ω → Pin 7; EW Yellow LED → 220Ω → Pin 8; EW Green LED → 220Ω → Pin 9
- Pedestrian Walk LED (green) → 220Ω → Pin 10
- Pedestrian Don't Walk LED (red) → 220Ω → Pin 11
- Buzzer positive leg → Pin 12; negative leg → GND

```
// Smart City - Traffic System
// Two sets of lights (North-South and East-West) run a timed sequence.
// A pedestrian button (with interrupt) can safely request a walk phase.

const int buttonPin = 2;       // Interrupt-capable pin on Uno
const int nsRed = 4, nsYellow = 5, nsGreen = 6;
const int ewRed = 7, ewYellow = 8, ewGreen = 9;
const int pedWalk = 10, pedDontWalk = 11;
const int buzzerPin = 12;

// Timing (ms) - tweak these to change how long each phase lasts
const unsigned long GREEN_TIME       = 6000;
const unsigned long YELLOW_TIME      = 2000;
const unsigned long ALL_RED_TIME     = 1000;
const unsigned long WALK_TIME        = 4000;
const unsigned long WALK_BLINK_TIME  = 2000;

enum State { NS_GREEN, NS_YELLOW, ALL_RED_A, PED_WALK_PHASE, PED_BLINK_PHASE, ALL_RED_B, EW_GREEN,
EW_YELLOW };
State currentState = NS_GREEN;

unsigned long stateStartTime = 0;
volatile bool pedestrianRequested = false;
volatile unsigned long lastInterruptTime = 0;

void enterState(State newState); // forward declaration

void setup() {
  pinMode(nsRed, OUTPUT); pinMode(nsYellow, OUTPUT); pinMode(nsGreen, OUTPUT);
  pinMode(ewRed, OUTPUT); pinMode(ewYellow, OUTPUT); pinMode(ewGreen, OUTPUT);
  pinMode(pedWalk, OUTPUT); pinMode(pedDontWalk, OUTPUT);
  pinMode(buzzerPin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);

  attachInterrupt(digitalPinToInterrupt(buttonPin), requestCrossing, FALLING);

  Serial.begin(9600);
  stateStartTime = millis();
  enterState(NS_GREEN);
}

void loop() {
  unsigned long elapsed = millis() - stateStartTime;

  switch (currentState) {
    case NS_GREEN:
```

```
      if (elapsed >= GREEN_TIME) enterState(NS_YELLOW);
      break;

    case NS_YELLOW:
      if (elapsed >= YELLOW_TIME) enterState(ALL_RED_A);
      break;

    case ALL_RED_A:
      if (elapsed >= ALL_RED_TIME) {
        if (pedestrianRequested) {
          enterState(PED_WALK_PHASE);
        } else {
          enterState(EW_GREEN);
        }
      }
      break;

    case PED_WALK_PHASE:
      if (elapsed >= WALK_TIME) enterState(PED_BLINK_PHASE);
      break;

    case PED_BLINK_PHASE:
      // Blink the don't-walk hand as a warning that time is running out
      if ((elapsed / 250) % 2 == 0) {
        digitalWrite(pedDontWalk, HIGH);
      } else {
        digitalWrite(pedDontWalk, LOW);
      }
      if (elapsed >= WALK_BLINK_TIME) {
        pedestrianRequested = false;
        enterState(EW_GREEN);
      }
      break;

    case EW_GREEN:
      if (elapsed >= GREEN_TIME) enterState(EW_YELLOW);
      break;

    case EW_YELLOW:
      if (elapsed >= YELLOW_TIME) enterState(ALL_RED_B);
      break;

    case ALL_RED_B:
      if (elapsed >= ALL_RED_TIME) enterState(NS_GREEN);
      break;
  }
}

void enterState(State newState) {
  currentState = newState;
  stateStartTime = millis();

  // Reset everything, then turn on only what this state needs
  digitalWrite(nsRed, LOW); digitalWrite(nsYellow, LOW); digitalWrite(nsGreen, LOW);
  digitalWrite(ewRed, LOW); digitalWrite(ewYellow, LOW); digitalWrite(ewGreen, LOW);
  digitalWrite(pedWalk, LOW); digitalWrite(pedDontWalk, LOW);
  noTone(buzzerPin);

  switch (newState) {
    case NS_GREEN:
```

```
      digitalWrite(nsGreen, HIGH); digitalWrite(ewRed, HIGH); digitalWrite(pedDontWalk, HIGH);
      break;
    case NS_YELLOW:
      digitalWrite(nsYellow, HIGH); digitalWrite(ewRed, HIGH); digitalWrite(pedDontWalk, HIGH);
      break;
    case ALL_RED_A:
      digitalWrite(nsRed, HIGH); digitalWrite(ewRed, HIGH); digitalWrite(pedDontWalk, HIGH);
      break;
    case PED_WALK_PHASE:
      digitalWrite(nsRed, HIGH); digitalWrite(ewRed, HIGH);
      digitalWrite(pedWalk, HIGH);
      tone(buzzerPin, 1000); // announce that it's safe to cross
      break;
    case PED_BLINK_PHASE:
      digitalWrite(nsRed, HIGH); digitalWrite(ewRed, HIGH);
      // pedDontWalk is blinked from inside loop()
      break;
    case EW_GREEN:
      digitalWrite(ewGreen, HIGH); digitalWrite(nsRed, HIGH); digitalWrite(pedDontWalk, HIGH);
      break;
    case EW_YELLOW:
      digitalWrite(ewYellow, HIGH); digitalWrite(nsRed, HIGH); digitalWrite(pedDontWalk, HIGH);
      break;
    case ALL_RED_B:
      digitalWrite(nsRed, HIGH); digitalWrite(ewRed, HIGH); digitalWrite(pedDontWalk, HIGH);
      break;
  }
}

void requestCrossing() {
  // Simple debounce: ignore repeated triggers within 300ms
  unsigned long now = millis();
  if (now - lastInterruptTime > 300) {
    pedestrianRequested = true;
    lastInterruptTime = now;
  }
}
```

Checks for understanding

- Why would using `delay()` for each phase make the pedestrian button unreliable?
- What does the debounce check inside `requestCrossing()` actually prevent?
- Why is there always an "all red" phase between NS and EW green — what would happen without it?

MISTAKE	FIX
Button press does nothing	Confirm it's wired to Pin 2 (interrupt pin) and to GND, and that `INPUT_PULLUP` plus `FALLING` are both used consistently
Both directions show green at once after edits	Check that `enterState()` always resets all LEDs LOW before setting the new state's outputs
Buzzer makes no sound	

Passive buzzers need `tone()` / `noTone()`, not `digitalWrite()` — confirm the sketch uses `tone(buzzerPin, 1000)`

---

3. Parking Garage — Ultrasonic Car Counter with FULL/OPEN Sign

Objectives

- Measure distance with an HC-SR04 ultrasonic sensor and understand the pulse-timing calculation behind it.
- Use sensor state transitions (not raw readings) to reliably count discrete events like a car passing.
- Drive a servo gate arm and a 4-digit display to build a small automated system with a real "capacity" rule.

Timing

- 0–10 min: Hook — demonstrate an HC-SR04 with the Serial Monitor open, waving a hand at different distances.
- 10–25 min: Explain how ultrasonic ranging works (send a pulse, time the echo, convert to distance) and why we need two sensors (entry and exit) to know direction.
- 25–30 min: Explain the counting problem: "if we just check distance every loop, one car will get counted 50 times as it passes — we need to detect the moment it arrives, not every reading."
- 30–80 min: Wire both ultrasonic sensors, the servo, the TM1637 display and the two sign LEDs.
- 80–120 min: Install libraries (TM1637Display), type and upload the sketch together, test entry/exit counting with a hand or small object.
- 120–135 min: Test capacity logic — fill the garage to MAX_SPOTS and confirm the sign switches to FULL and the gate stops opening.
- 135–140 min: Attach a popsicle-stick arm to the servo horn with hot glue and mount the module for the city board.

Step-by-step

1. Show the HC-SR04, explain TRIG (sends the "ping") and ECHO (times the return).
2. Walk through the maths on the whiteboard: distance = (time × speed of sound) ÷ 2, and why the sketch uses 0.0343 cm/µs.

3. Discuss why a single sensor can't tell entering from exiting — introduce the two-sensor entry/exit design.
4. Guide wiring in stages: get one ultrasonic sensor printing distance to Serial Monitor before adding the second, then the servo, then the display, then the LEDs.
5. Install the TM1637Display library together via Library Manager (Sketch → Include Library → Manage Libraries → search "TM1637").
6. Explain the "edge detection" logic in code: a car is only counted the moment `carAtEntryNow` becomes true after being false — not on every loop it stays close.
7. Test thoroughly: pass a hand through the entry sensor 3 times, confirm count goes up exactly 3, then through exit, confirm it goes back down.
8. Test the FULL condition by "parking" 5 cars, confirming the sign changes and the gate stops responding to entry.

Build detail

Materials: Arduino Uno, breadboard, 2x HC-SR04 ultrasonic sensor, 1x SG90 micro servo, 1x TM1637 4-digit display module, 1x green LED, 1x red LED, 2x 220Ω resistor, jumper wires, popsicle stick, hot glue.

Wiring (component → pin):

- Entry HC-SR04: TRIG → Pin 2; ECHO → Pin 3; VCC → 5V; GND → GND
- Exit HC-SR04: TRIG → Pin 4; ECHO → Pin 5; VCC → 5V; GND → GND
- Servo (gate arm): signal wire → Pin 6; power → 5V; GND → GND
- TM1637 display: CLK → Pin 7; DIO → Pin 8; VCC → 5V; GND → GND
- OPEN sign LED (green) → 220Ω → Pin 9
- FULL sign LED (red) → 220Ω → Pin 10

```
// Smart City - Parking Garage
// Two ultrasonic sensors count cars in and out. A servo gate arm lifts on
// entry (if there is space), a 4-digit display shows the live count, and
// two LEDs act as an OPEN/FULL sign.
// Requires the "TM1637Display" library (by Avishay Orpaz) via Library Manager.

#include
#include

// Ultrasonic sensor pins
const int entryTrig = 2, entryEcho = 3;
const int exitTrig = 4, exitEcho = 5;
```

```
// Servo (gate arm)
const int servoPin = 6;
Servo gate;

// 4-digit display (shows current car count)
const int clkPin = 7, dioPin = 8;
TM1637Display display(clkPin, dioPin);

// Sign LEDs
const int openLed = 9;
const int fullLed = 10;

const int MAX_SPOTS = 5;
int carCount = 0;

const int DETECT_DISTANCE = 8; // cm - car counts as "at the sensor" if closer than this

bool carWasAtEntry = false;
bool carWasAtExit = false;

void setup() {
  pinMode(entryTrig, OUTPUT); pinMode(entryEcho, INPUT);
  pinMode(exitTrig, OUTPUT);  pinMode(exitEcho, INPUT);
  pinMode(openLed, OUTPUT);
  pinMode(fullLed, OUTPUT);

  gate.attach(servoPin);
  gate.write(0); // gate starts closed (down)

  display.setBrightness(5);
  updateDisplay();
  updateSign();

  Serial.begin(9600);
}

long readDistanceCm(int trigPin, int echoPin) {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long duration = pulseIn(echoPin, HIGH, 20000); // 20ms timeout (~3.4m max range)
  if (duration == 0) return 999; // no echo received = nothing in range
  return duration * 0.0343 / 2;  // convert time to distance in cm
}

void loop() {
  long entryDist = readDistanceCm(entryTrig, entryEcho);
  long exitDist  = readDistanceCm(exitTrig, exitEcho);

  bool carAtEntryNow = entryDist < DETECT_DISTANCE;
  bool carAtExitNow  = exitDist  < DETECT_DISTANCE;

  // Only count on the transition from "no car" to "car detected" -
  // this stops one passing car from being counted dozens of times.
  if (carAtEntryNow && !carWasAtEntry) {
    if (carCount < MAX_SPOTS) {
      carCount++;
```

```
      Serial.println("Car entered");
      openGateBriefly();
      updateDisplay();
      updateSign();
    } else {
      Serial.println("Garage FULL - entry refused");
    }
  }

  if (carAtExitNow && !carWasAtExit) {
    if (carCount > 0) {
      carCount--;
      Serial.println("Car exited");
      updateDisplay();
      updateSign();
    }
  }

  carWasAtEntry = carAtEntryNow;
  carWasAtExit = carAtExitNow;

  delay(100); // small delay keeps ultrasonic readings stable
}

void openGateBriefly() {
  gate.write(90);   // lift the arm
  delay(800);       // give the car time to pass under (demo timing)
  gate.write(0);    // lower the arm
}

void updateDisplay() {
  display.showNumberDec(carCount, false);
}

void updateSign() {
  bool isFull = (carCount >= MAX_SPOTS);
  digitalWrite(fullLed, isFull ? HIGH : LOW);
  digitalWrite(openLed, isFull ? LOW : HIGH);
}
```

Checks for understanding

- Why does the code check for a *transition* (car wasn't there, now it is) instead of just checking "is a car close right now"?
- What happens in the code if `carCount` is already at `MAX_SPOTS` and another car arrives at the entry sensor?
- Why do we need two ultrasonic sensors instead of one for this system?

MISTAKE	FIX
Car count jumps by more than 1 for a single pass	Object passed too slowly through the beam or DETECT_DISTANCE is too large; check the edge-detection logic is intact and not accidentally removed

Servo jitters or doesn't move	Servo may be underpowered from the Arduino 5V pin with other devices attached; if unstable, power servo from a separate 5V supply sharing GND with the Arduino
Display shows nothing or garbage	Check CLK/DIO are not swapped, and confirm the TM1637Display library is installed and included at the top of the sketch

4. Weather Station — DHT11 + LDR on an LCD

Objectives

- Read temperature and humidity from a DHT11 sensor and combine it with an LDR light reading to describe "weather".
- Drive a 16x2 I2C LCD to display rotating screens of live data.
- Practise non-blocking timing with `millis()` to separate "how often we read sensors" from "how often we change the screen".

Timing

- 0–10 min: Hook — pass around the DHT11, ask what two things it can measure, and discuss why weather stations combine multiple sensors.
- 10–25 min: Explain the I2C bus concept simply ("SDA and SCL let many devices share just two wires") and why the LCD only needs 4 wires with the I2C backpack.
- 25–70 min: Wire the DHT11, LDR voltage divider, and I2C LCD.
- 70–75 min: Install libraries together (DHT sensor library, Adafruit Unified Sensor, LiquidCrystal_I2C) via Library Manager.
- 75–110 min: Type and upload the sketch; if the LCD shows nothing, run a quick I2C scanner sketch to confirm the address.
- 110–130 min: Test by breathing near the DHT11 (raises humidity briefly) and covering the LDR (changes sky reading); confirm both screens rotate correctly.
- 130–140 min: Mount the LCD and sensors into a small cardboard "weather station" housing for the city board.

Step-by-step

1. Introduce the DHT11 and clarify it only updates roughly once per second — explain why the code won't read it every single loop.
2. Draw the I2C bus on the whiteboard: SDA (A4) and SCL (A5) shared by the LCD, only 4 wires total needed (VCC, GND, SDA, SCL).

3. Guide wiring: DHT11 first, test raw values on Serial Monitor, then LDR, then the LCD last (since I2C issues are easiest to debug once the rest works).
4. If the LCD backlight is on but no text shows, run this quick address-scanning sketch as a checkpoint before continuing:

```
#include

void setup() {
  Serial.begin(9600);
  Wire.begin();
  Serial.println("Scanning I2C bus...");
  for (byte addr = 1; addr < 127; addr++) {
    Wire.beginTransmission(addr);
    if (Wire.endTransmission() == 0) {
      Serial.print("Found device at 0x");
      Serial.println(addr, HEX);
    }
  }
}

void loop() {}
```

5. Update the LCD address constant in the main sketch to match whatever the scanner found (commonly 0x27 or 0x3F).
6. Type and upload the full weather station sketch together, explaining the two-screen rotation logic.
7. Test edge cases: cup hands around the DHT11 and breathe to raise humidity above 70% and confirm the "Rain likely" message appears.

Build detail

Materials: Arduino Uno, breadboard, DHT11 sensor module, LDR, 10kΩ resistor, 16x2 I2C LCD (address 0x27 or 0x3F), jumper wires, small cardboard housing.

Wiring (component → pin):

- DHT11 module: VCC → 5V; GND → GND; DATA → Pin 7
- LDR leg 1 → 5V; LDR leg 2 → A0 and → one leg of 10kΩ resistor → GND
- LCD I2C backpack: VCC → 5V; GND → GND; SDA → A4; SCL → A5

```
// Smart City - Weather Station
// Reads temperature + humidity from a DHT11 and light level from an LDR,
// then rotates two info screens on a 16x2 I2C LCD.
// Requires libraries: "DHT sensor library" (Adafruit), "Adafruit Unified Sensor",
// and "LiquidCrystal_I2C".

#include
```

```
#include
#include

#define DHTPIN 7
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

LiquidCrystal_I2C lcd(0x27, 16, 2); // change 0x27 to 0x3F if the I2C scanner found that instead

const int ldrPin = A0;

unsigned long lastReadTime = 0;
const unsigned long READ_INTERVAL = 2500; // DHT11 only refreshes every ~1-2s; this is safe

unsigned long lastScreenChange = 0;
const unsigned long SCREEN_INTERVAL = 3000;
int screenIndex = 0; // 0 = temp/humidity, 1 = sky/forecast

float temperature = 0;
float humidity = 0;
int lightLevel = 0;

void setup() {
  Serial.begin(9600);
  dht.begin();
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Weather Station");
  lcd.setCursor(0, 1);
  lcd.print("Starting up...");
  delay(1500);
  lcd.clear();
}

String skyCondition(int level) {
  if (level > 700) return "Bright/Sunny";
  if (level > 300) return "Cloudy";
  return "Dark";
}

void loop() {
  unsigned long now = millis();

  // Take new sensor readings only every READ_INTERVAL
  if (now - lastReadTime >= READ_INTERVAL) {
    lastReadTime = now;
    float h = dht.readHumidity();
    float t = dht.readTemperature();
    if (!isnan(h) && !isnan(t)) {
      humidity = h;
      temperature = t;
    } else {
      Serial.println("DHT11 read failed - check wiring");
    }
    lightLevel = analogRead(ldrPin);
  }

  // Flip between two info screens every SCREEN_INTERVAL
  if (now - lastScreenChange >= SCREEN_INTERVAL) {
```

```
    lastScreenChange = now;
    screenIndex = 1 - screenIndex;
    lcd.clear();
  }

  if (screenIndex == 0) {
    lcd.setCursor(0, 0);
    lcd.print("Temp: ");
    lcd.print(temperature, 1);
    lcd.print((char)223); // degree symbol
    lcd.print("C");
    lcd.setCursor(0, 1);
    lcd.print("Humidity: ");
    lcd.print(humidity, 0);
    lcd.print("%");
  } else {
    lcd.setCursor(0, 0);
    lcd.print("Sky: ");
    lcd.print(skyCondition(lightLevel));
    lcd.setCursor(0, 1);
    if (humidity > 70) {
      lcd.print("Rain likely");
    } else {
      lcd.print("Clear skies");
    }
  }
}
```

Checks for understanding

- Why does the sketch only read the DHT11 every 2.5 seconds instead of every loop?
- What do SDA and SCL do, and why does the LCD only need those two data wires?
- How would you change the code to add a third screen showing raw light level as a number?

MISTAKE	FIX
LCD backlight on but no text	Run the I2C scanner sketch to find the correct address and update it in the main sketch
Temperature/humidity always reads "nan"	Check DATA pin matches `DHTPIN`, and that the DHT11 module has power; DHT11 needs a brief settle time after power-up
Screen flickers constantly	Expected with continuous printing; if excessive, only call `lcd.clear()` on screen change (already done) and avoid clearing inside the sensor-read block

Assessment & Showcase

Assessment across the day is formative and observational rather than test-based. For each team, the facilitator tracks three things on a simple checklist per subsystem: (1) the circuit is wired correctly and matches the wiring list, (2) the code runs without errors and the intended behaviour is demonstrable on request (e.g., "make it think it's night" or "make a car arrive"), and (3) at least one team member can explain, in their own words, how the sensor and the code work together to produce the behaviour. A team is considered "done" when all three are true and their module is safely mounted and ready to move to the shared board.

During the last 45–75 minutes of the day, run the Integration phase: bring all four modules to the shared city board (powered independently via USB power bank or the shared USB hub, since they do not need to share electronics — only the physical layout). Have each team place their module on its marked zone (street lights along the roads, traffic lights at the intersection, parking garage at the parking bay, weather station near the "town square"), and do a full walkthrough: dim the room lights to trigger the street lights, run a full traffic light cycle and press the pedestrian button, drive a toy car through the parking sensors, and read the weather station's rotating screens aloud.

Close with a 20–30 minute Showcase: each team presents their subsystem to the whole group (30–60 seconds each) explaining what sensor they used, what decision their code makes, and one thing they'd improve with more time. Take the official group photo of the assembled city board with all four systems visibly running (dim the lights briefly for a dramatic "night mode" shot with the street lights and traffic lights lit). "Done" for the day means every team can demonstrate their module live, on request, in under a minute, without facilitator help.

Extension & Home Challenges

- **Easy:** Street Lights team — replace the abrupt on/off with a smooth fade using `analogWrite()` so the lamps gradually brighten as it gets darker instead of switching instantly.
- **Medium:** Traffic System team — add a second pedestrian crossing on the other road (a second button and pair of pedestrian LEDs), and update the state machine so either crossing can be requested independently.

- **Hard:** Parking Garage team — add a second exit gate with its own servo, and modify the display logic to show "FULL" as text (scrolling across the 4-digit display or via the Weather Station's spare LCD) instead of just lighting a red LED, including handling the edge case where a car arrives at both entry and exit at the same instant.